数字生命：一个可测量的生命定义

工程卷（持续更新）

> 何淇 · 与 DeepSeek（AI）协作

> 数字生命系列 · 工程卷（持续更新）

> 日期：2026-11-08

> 本卷为系列合集的分卷版；完整版见系列总合集。各篇单独 DOI 保留，以总合集与各篇发布记录为准。

卷首简介

本卷收录工程部分：门控与稀疏执行（汇总）、冻结子模型的两组最小实验、数字蚊子的最小工程实现，以及工程笔记（持续更新板块）。

和其他卷不同，**工程卷是持续更新的卷**：以后新的工程实验、失败记录、判断修正，都会继续加进"工程笔记"部分，并按新版本重新发布本卷。

v3 起新增：工程笔记 02《一个自学习的 LLM 模型——冻结、成长与切换》——在冻结底座的 GPT-2 上做 6+6 加层实验，记录"只加层能学新东西、但旧任务退化"与"加门+旧门保底后旧任务不丢、新任务学到、门可自选"两组结果。

v4 起新增：工程笔记 03《门的复用与组合》——门不累积（交替会话累积臂≈重置臂）；联合训一个门是折中（6.24/40.33）；共享门+每任务小偏移接近专精（4.58/27.27）。

v5 起新增：论文-49《门：结构上的可塑接口》；工程笔记 04《组合任务的门》；工程笔记 05《门的极限》（真模型 Qwen2.5 0.5B/1.5B/3B 适配：新领域 PPL 降 61~64%）。

v6 起（2026-11-13）：门相关篇目汇总为两篇——论文-50《门：从可塑接口到重配比》（定义、三档粒度、能做什么/不能做什么、阈值测量、门当诊断的负结果、与 ReFT/CeRA 的关系）与论文-51《门与模型一起训练：门的稀疏性是结构选择》（联合训练 44%~50% 层、代价 0~7%；预训练+LoRA 压不动；稀疏执行等价；MoD 路由与墙钟负结果）。原论文-44、论文-49 与工程笔记 03/04/05 保留 DOI，视为被这两篇汇总取代，不再列入本卷目录。另新增工程笔记 08《结构作为子空间》。

v7 起（2026-11-14）：新增论文-52《上下文卸载与局部适配——一个多模型拼接 + 独立记忆通道的思路》（思路草稿 v1：tiny/合成实验与负结果；不声称优于现有方法）。

目录

第四部分 执行器工程（论文）

- 第二十四篇　冻结子模型：接口、协调与分层——两组最小实验（待复核）

- 第二十五篇　冻结模块与可训练大脑：数字蚊子的最小工程实现（待复核）

- 第二十六篇　门：从可塑接口到重配比

- 第二十七篇　门与模型一起训练：门的稀疏性是结构选择

- 第二十八篇　上下文卸载与局部适配：一个多模型拼接 + 独立记忆通道的思路（思路草稿）

第五部分 工程笔记（过程记录）

- 第二十九篇　工程笔记 01：门控与模块化的失败、修正与判断（持续更新）

- 第三十篇　方向观察 01：加层、回放与蒸馏——大模型睡眠巩固（待复核）

- 第三十一篇　工程笔记 02：一个自学习的 LLM 模型——冻结、成长与切换

- 第三十二篇　工程笔记 07：维度门槛——给够维度才能拟合，门跨不过去

- 第三十三篇　工程笔记 08：结构作为子空间——一个可测的版本

第二十四篇 · 冻结子模型：接口、协调与分层——两组最小实验（待复核）

> 单独 DOI：https://doi.org/10.5281/zenodo.23859913

> 何淇 · 与 DeepSeek 协作

> 数字生命系列 · 实验篇（内部编号第 45 篇）

> 日期：2026-11-01

> 状态：最小实验观察——单种子、小规模；不作方法宣称。

摘要

本篇回答一个问题：**已经训练好、被冻结的子模型，能不能通过接口组合起来，形成新的结构？**

我们做了两组互补的最小实验：

实验一（并列组合）：两个独立训练的 6 层字符级模型——A 学"正常文本的下一个字符"，B 学"凯撒位移（+1）文本的下一个字符"。把 A、B 冻结，在它们之间加可训练的总控接口，支持四种模式：A 单独、B 单独、A→B（正常输入、位移输出）、B→A（位移输入、正常输出）。

结果：

- 直接首尾拼接（接口为恒等）：PPL 62.84（崩）；

- 加可训练总控：A→B 5.94，B→A 5.67；

- 原生 12 层模型（同任务上界）：5.41；

- A 单独 6.03，B 单独 6.31。

实验二（分层组合）：一个最小趋化环境（一维圈形跑道、食物消耗与再生、能量与死亡）。感知模块 M1（监督训练）、适应模块 M2（监督训练）冻结，决策层 D 用强化学习在环境里训练。

结果：

- 随机策略：回报 -1.34、平均存活 66 步、死亡率 97%；

- 只用原始传感器 + D：回报 -1.40、存活 45 步、死亡率 98%（学不出来）；

- M1 + D：回报 0.494、存活 183 步、死亡率 10%；

- M1 + M2 + D：回报 0.505、存活 184 步、死亡率 9%。

结论（观察，不是定论）：

1. 冻结子模型不能直接拼接，需要可训练的接口/总控；

2. 加入总控后，并列组合可以接近原生模型（5.94/5.67 vs 5.41）；

3. 分层组合中，冻结的下层特征对上层决策是决定性的（M1 把死亡率从 98% 降到 10%）；

4. 适应模块 M2 在本实验中只带来微小提升（0.494 → 0.505），其价值尚未被环境充分"逼出来"。

一、问题与背景

1.1 为什么要"冻结子模型组合"

训练一个模型昂贵；如果能力可以被压缩进小的、可冻结的子模型，然后把它们组合起来，就能：

- 复用已有能力（不用重训）；

- 替换单个模块而不动其他部分；

- 逐步构建更复杂的结构（分层/模块化）。

但冻结也意味着**不能靠继续训练来补偿**：接口必须自己完成协调。这正是本篇要测的。

1.2 与已有工作的关系

我们不声称新方法。相邻工作包括：

- Model stitching：冻结两半网络，用小缝合层连接，用于表示相似性研究；

- 任务向量/功能向量：能力在权重或激活空间中的可迁移表示；

- 冻结专家组合：多个冻结模块 + 可训练接口/路由（如 Bicameral Model 用可训练接口双向耦合两个冻结语言模型）。

我们的实验是这些思想在小规模上的最小化验证：**接口是否必要、接口能补回多少、分层冻结是否可行。**

1.3 术语

- **冻结子模型**：训练完成后参数不再更新的模型。

- **总控/接口**：插在两个冻结子模型之间的可训练小模块，负责表示空间对齐（本实验中是两个 2 层 MLP）。

- **并列组合**：两个子模型能力不同、通过接口互相衔接（实验一）。

- **分层组合**：下层冻结模块提供特征，上层可训练模块做决策（实验二）。

二、实验一：并列组合（两个冻结语言模型 + 总控）

2.1 设置

- 模型：两个 tiny GPT-2 风格字符级模型（各 6 层、隐藏维度 256、8 头、字符级词表 65）。

- 数据：tinyshakespeare 字符级，序列长度 128。

- 任务：

- A：正常文本的下一个字符；

- B：凯撒位移（+1）文本的下一个字符。

- 训练：A、B 各自从零训练 1200 步（batch 16，lr 3e-4），训练后冻结。

- 总控：每个方向一个 2 层 MLP（256→256→256，GELU），初始近似恒等；训练时只更新总控，A、B 冻结。

- 对照：

- 恒等接口（不训练）；

- 原生 12 层模型（同任务，1500 步），作为"上界"；

- A 单独、B 单独的 PPL。

2.2 模式定义

```

A-only：输入 → A 的嵌入 → A 的 6 层 → A 的 ln_f + lm_head

B-only：输入 → B 的嵌入 → B 的 6 层 → B 的 ln_f + lm_head

A→B： 输入 → A 的嵌入 → A 的 6 层 → C_AB → B 的 6 层 → B 的 ln_f + lm_head

B→A： 输入 → B 的嵌入 → B 的 6 层 → C_BA → A 的 6 层 → A 的 ln_f + lm_head

```

2.3 结果

| 设置 | 验证 PPL |

|---|---|

| A 单独（正常文本） | 6.03 |

| B 单独（位移文本） | 6.31 |

| 原生 12 层（翻译任务上界） | 5.41 |

| A→B 直接拼（恒等接口） | **62.84** |

| A→B + 训练的总控 | **5.94** |

| B→A + 训练的总控 | **5.67** |

2.4 观察

1. **恒等接口完全不行**（62.84）：两个独立训练模型的隐藏表示空间不同，直接首尾相接等于把一种"语言"喂给只懂另一种"语言"的层。

2. **可训练总控能补回来**：A→B 5.94、B→A 5.67，接近原生 12 层上界 5.41，也接近两个子模型各自的水平（6.0~6.3）。

3. **总控非常小**：两个方向共约 26 万参数，相对两个子模型（各约 500 万参数）约 2.6%。

4. **方向基本对称**：A→B 与 B→A 差距很小（5.94 vs 5.67），本实验中未出现明显不对称。

2.5 局限

- 字符级 tiny 模型、凯撒位移是极简单任务；

- 原生 12 层上界只训练 1500 步，可能未完全收敛；

- 模式是离散切换（四次独立模式），不是"同时双向耦合"；

- 单种子。

三、实验二：分层组合（趋化环境中的冻结感知/适应 + 可训练决策）

3.1 环境

最小趋化环境（纯函数模拟器，CPU 上运行）：

- 一维圈形跑道，64 格；

- 食物场 f[i] ∈ [0,1]：初始 2~3 个高斯热点；被吃后减少；每步按 0.0015 再生；

- 智能体：位置 p、能量 e（初始 0.5，上限 0.8）；

- 每步消耗 0.035 能量；

- 吃当前格食物：最多 0.15×0.6 的补充；

- e ≤ 0 → 死亡，回合结束；

- 观测：前后各 2 格平均食物量（加高斯噪声 0.02）；

- 动作：左移 / 右移 / 停留；

- 每回合最多 200 步；

- 奖励：能量变化 + 0.002（存活）；死亡额外 -1。

3.2 模块

```

M1 感知（监督训练，冻结）：

· 输入：左右传感器读数（2 维）

· 输出：哪边食物多（2 类）

· 标签：模拟器中的真实左右食物量比较（精确可算）

· 训练：4000 个随机位置样本

M2 适应（监督训练，冻结）：

· 输入：最近 6 步的传感器均值（6 维）

· 输出：背景食物水平（标量）

· 标签：模拟器中的全场平均食物量

· 训练：3000 个序列

D 决策（可训练）：

· 输入：原始传感器（2）+ M1 输出（2）+ M2 输出（1）

· 输出：3 个动作的 logits

· 训练：强化学习（REINFORCE + 回报归一化），2500 回合

· M1、M2 冻结，只更新 D
```

```

**3.3** 结果

| 策略 | 回报 | 平均存活步数 | 死亡率 |

|---|---|---|---|

| 随机策略 | -1.34 | 66 | 0.97 |

| 只用原始传感器 + D | -1.40 | 45 | 0.98 |

| M1 + D | 0.494 | 183 | 0.10 |

| M1 + M2 + D | 0.505 | 184 | 0.09 |

**3.4** 观察

1. **环境有真实压力**：随机策略 97% 会饿死，平均只活 66 步。

2. **只有原始传感器时决策层学不出来**：回报 -1.40、死亡率 98%，与随机策略相当。单步带噪观测不足以让一个小 MLP 用策略梯度学到趋化。

3. **冻结的感知模块是决定性的**：加入 M1 后，回报从负转正（0.494）、存活步数 183、死亡率降到 10%。

4. **适应模块 M2 只带来微小提升**：0.494 → 0.505。可能原因：本环境背景变化缓慢，M2 提供的信息增量有限；也可能 M2 的监督目标（全场均值）与决策需要不完全一致。

5. **工程上可行**：全部为小 MLP + CPU，几分钟完成训练与评测。

**3.5** 局限

- 一维、离散格、简化食物动态；

- M1 与 M2 的功能在一维设定下有重叠，二者的独立贡献未被完全分离；

- RL 只训练 2500 回合，可能未收敛；

- 单种子。

## 四、讨论
```

4.1 为什么恒等接口会失败

[解释] 两个独立训练模型各自形成了自己的隐藏表示坐标（尺度、方向、特征基）。A 的最后一层输出是"给 A 的输出层用"的表示，不是 B 的输入层期望的表示。直接相接等于跨语言通信而不做翻译，失败是预期的。

4.2 为什么小总控能补回来

[解释] 总控只需要做"表示空间之间的映射"，不需要重新学习任务。映射是一个低维线性/非线性变换，参数量远小于子模型，但足以把 A 的表示搬到 B 的输入分布附近。实验中 2.6% 的参数补回了 ~90% 的差距（62.84 → 5.94，上界 5.41）。

4.3 为什么分层组合中下层冻结是决定性的

[解释] 决策层只看到观测与下层特征。如果观测本身信息不足（单步带噪传感器），策略梯度很难从稀疏奖励里学到有效行为；而冻结的感知模块把"哪边食物多"这个判别特征直接提供给决策层，等价于给了一个好的状态表示。这与"预训练特征 + 线性探测/微调头"的常见经验一致。

4.4 关于**"感知与适应是否重复"**

[事实] 一维设定下 M1 与 M2 的功能确有重叠（都在处理传感器历史/差异）。本实验没有分离出"历史信息"与"感知特征"各自的贡献。要回答这个问题，需要加入"raw_hist"（原始历史但无 M1/M2）与"循环决策层"两组对照。

五、局限（总）

- 两个实验均为单种子、极小规模（字符级 tiny 模型 / 一维环境）；

- 实验一的"上界"模型训练步数少于子模型，比较不严格；

- 实验二的 RL 训练量有限，M2 的收益可能被低估；

- 未做"同时双向耦合"（lockstep）与"逐层冻结生长"的实验；

- 未验证真实规模模型上的可迁移性。

六、结论

1. **冻结子模型不能直接拼接**；需要一个可训练的接口/总控来完成表示对齐（实验一：62.84 → 5.94）。

2. **小总控（约 2.6% 参数）可以补回大部分损失**，使并列组合接近原生模型（5.94/5.67 vs 5.41）。

3. **分层组合成立**：冻结的感知模块 + 可训练决策层，在趋化环境中把死亡率从 98% 降到 10%，回报由负转正。

4. **适应模块的价值未显现**（0.494 → 0.505），需要更"需要适应"的环境来检验。

5. 工程实现无特殊门槛：小 MLP + CPU 即可完成，接口与模块都可以冻结/替换。

附：标注

- **[事实]** 第二、三节全部 PPL、回报、存活步数、死亡率（来自运行结果）。

- **[解释]** 第四节的分析。

- **[推测]** "适应模块在更难环境中会有价值"——未验证。

- **[不声称]** 不声称新方法；不声称优于原生模型；不声称可扩展到真实规模。

- 相邻工作：model stitching、任务/功能向量、冻结专家组合（如 Bicameral Model）。

第二十五篇 · 冻结模块与可训练大脑：数字蚊子的最小工程实现（待复核）

> 单独 DOI：https://doi.org/10.5281/zenodo.23890632

> 何淇 · 与 DeepSeek 协作

> 数字生命系列 · 执行器工程（内部编号第 46 篇）

> 日期：2026-11-03

> 状态：最小工程实现与观察——单种子、小规模；不作方法宣称。

> 说明：本篇与论文-45《冻结子模型：接口、协调与分层》是同一原理的两种形态：论文-45 做的是两个冻结语言模型之间的拼接与协调；本篇做的是"冻结的感知/执行模块 + 可训练决策大脑"。

摘要

本篇回答一个工程问题：**如果感知和执行都交给冻结模块，只让"大脑"可训练，一个小系统能学会生存行为吗？**

我们做了一个"数字蚊子"的最小实现：

- **冻结的眼睛**：12 条 360° 射线，返回归一化距离（固定算法）；

- **冻结的翅膀**：转向动力学 + 速度一阶低通（固定算法）；

- **可训练的大脑**：64×64 两层 MLP，输出（转向，推力）；

- **环境**：二维场地，障碍物定期从边缘生成、朝蚊子当前位置飞来，碰到即死；

- **训练**：进化策略（ES）——本质就是"存活→参数保留→下一代"的演化。

主结果：

| 配置 | 平均存活步数 | 中位存活 | 被撞率 |

|---|---|---|---|

| 完整（冻结眼 + 冻结翅 + 演化脑） | **316.9** | 400（跑满） | 42.7% |

| 无眼睛（只有速度，盲飞） | 68.0 | 66 | 100% |

结论（观察，不是定论）：

1. 冻结的感知模块是决定性的：有眼睛能学会左右闪躲、平均活 317 步；没有眼睛几乎必死（68 步、100% 被撞）。

2. 本篇用固定算法作为"冻结模块"的最简形式；对上层可训练部分来说，它与"训练后冻结的网络"同样是"不可改、只能使用"的映射（一般情形的对照是后续工作）。

3. 演化策略（ES）在这类小系统上比策略梯度更稳定：早期的 REINFORCE 版本会学出"保守不动"或"自杀"策略。

4. 任务设计比算法更关键：我们先做"追目标 + 避障"，学会的是"躺平"或"冲锋"；换成"障碍冲向我、只能左右躲"后，行为才干净地出现。

一、问题与背景

1.1 冻结模块能不能组合、复用

论文-45 已经观察过：两个独立训练的语言模型冻结后直接拼接会崩（PPL 62.8），加一个可训练的小总控能恢复（5.94）；分层组合中，冻结的感知模块 + 可训练决策层能在趋化环境里把死亡率从 98% 降到 10%。

那组实验里的"冻结模块"都是训练好的神经网络。本篇的自然延续问题是：**冻结的（有参数的）模块 + 一个很小的可训练大脑，能不能在更接近"感知—动作"的闭环里学会生存行为？**

为了把问题压到最小，本篇的冻结模块用固定算法实现（射线检测与低通动力学）。它们是"冻结模块"的最简形式：参数（这里是算法常数）固定、不参与训练。更一般的情形——训练后冻结的神经网络——是后续工作；本篇不主张"固定算法"与"训练后冻结"在所有方面等价，只主张：对上层可训练部分来说，两者都是"不可改、只能使用"的映射。

1.2 生物原型：蚊子

蚊子是很好的原型：神经元少、视觉发达、时间尺度快（翅膀每秒几百次、反应几十毫秒）。它没有时间"想"每一次拍翅——快速回路必须是硬连线的（冻结），只有"往哪飞、躲谁"这类慢决策交给可学习/可演化的部分。

所以"数字蚊子"的结构很自然：

```
眼睛（冻结）→ 大脑（可训练/可演化）→ 翅膀（冻结）
```

1.3 两种"冻结"的来源

本篇使用"冻结模块"这个说法时，包含两种来源：

- **先天规则型**：固定算法（射线检测、低通动力学），参数固定、不参与训练；

- **训练后冻结型**：先训练一个小网络，再把参数冻结。

对上层可训练部分来说，这两种模块是等价的：都是"参数不再更新、只能使用"的映射。

二、实验设计

2.1 环境

- 二维场地 $[-1, 1]^2$；

- 蚊子：位置、朝向、速度；出生在中心附近；

- 障碍物：每 12 步从边缘生成一个，半径 0.025–0.055，朝蚊子当前位置飞行，速度 0.018–0.03/步；同屏最多 10 个；

- 碰撞（距离 < 障碍半径 + 0.02）→ 死亡，回合结束；

- 每回合最多 400 步，活满算"存活"。

2.2 冻结模块

眼睛（冻结）：

- 12 条 360° 射线，对每个障碍做射线-圆求交，取最近距离；

- 输出 12 维归一化距离（除以最大量程 0.8），再拼接当前速度（除以最大速度），共 13 维；

翅膀（冻结）：

- 转向：朝向 += 转向指令 × 0.35 弧度/步；

- 速度：一阶低通，speed += 0.25 × (目标速度 − speed)，最大速度 0.05/步；

2.3 可训练的大脑

- 输入：13 维（眼睛输出 + 速度）；

- 结构：Linear(13, 64) → Tanh → Linear(64, 64) → Tanh → Linear(64, 2) → Tanh；

- 输出：转向（[-1, 1]）、推力（[0, 1]）。

2.4 训练：进化策略（**ES**）

- 群体 32；每代对每个个体加高斯噪声（σ=0.06），确定性评估 2 个回合；

- 适应度 = 平均回报 + 2 × 平均存活步数；

- 按排名加权更新参数（OpenAI-ES 风格），学习率 0.05，共 180 代。

选 ES 而不是策略梯度的原因见 4.3：REINFORCE 版本不稳定，会收敛到"保守不动"或"自杀"。

2.5 奖励

- 每步存活：+0.02；

- 距离惩罚：12 条射线中，距离归一化值低于 0.35 的，按"越近罚越重"累加（权重 8×超出量）；

- 碰撞：−100，回合结束；

- 活满 400 步：+0.5。

三、结果

3.1 主结果

| 配置 | 平均存活步数 | 中位存活 | 被撞率 |

|---|---|---|---|

| 完整（冻结眼 + 冻结翅 + 演化脑） | **316.9** | 400（跑满） | 42.7% |

| 无眼睛（只有速度） | 68.0 | 66 | 100% |

（每种配置评估 300 个回合；单种子训练。）

3.2 观察

1. **冻结的眼睛是决定性的**：有眼睛的个体学会了左右闪躲，平均活 317 步，接近一半回合能撑满 400 步上限；没有眼睛的个体只会盲飞，平均 68 步、100% 被撞。

2. **小脑可以很小**：可训练部分只有 13→64→64→2 的两层 MLP，约 5 千参数；学习只发生在这一小块里。

3. **冻结模块确实"可用但不可改"**：大脑只能学怎么读眼睛、怎么驱动翅膀，不能改射线或动力学；这和"先天回路 + 可塑决策"的结构一致。

3.3 一个反例（先做的版本）

在换成"躲避"任务之前，我们做过"追目标 + 避障"的版本：

| 配置 | 到达率 | 撞障碍 | 撞墙 | 超时 |

|---|---|---|---|---|

| 完整（有眼睛） | 33% | 8.7% | 7% | 51% |

| 无眼睛 | 80.7% | 19.3% | 0% | 0% |

有眼睛反而更差：它看到障碍后变得保守，大量回合超时；没有眼睛的个体径直冲向目标，反而到达率高。

这个反例说明：**任务设计比算法更关键。** 在"追目标"任务里，眼睛提供了"危险的信号"，但没有提供"绕过去"的明确梯度；在"躲避"任务里，生存本身就是唯一目标，眼睛的信息才直接变成行为。

四、讨论

4.1 "冻结"的两种来源是等价的

本篇的眼睛和翅膀是固定算法（先天规则型），不是训练后冻结的网络。但对上层大脑来说，它们与"训练后冻结的网络"没有区别：参数不更新、行为固定、只能被使用。所以论文-45 的结论（冻结模块 + 可训练接口可以组合）在这里以另一种形态成立：**冻结组件 + 可训练的上层适配**。

4.2 演化与学习

ES 的更新规则（采样→按适应度加权→保留）本身就是"存活→参数保留→下一代"的演化模拟。这与之前的 RL 实验不同：RL 是个体在一生中学习；ES 是群体跨代演化。生物两者都有：先天回路（冻结）、一生学习（可塑）、代际遗传（演化）。

4.3 为什么用 **ES**

早期我们用 REINFORCE 训练策略，遇到两个问题：

- 单步回合时回报方差为 0，归一化产生 NaN；

- 更重要的是，策略梯度会学到两类退化解：

- **保守不动**：不动就不会靠近障碍、不会被罚，但也不前进；

- **自杀**：快速结束回合，避免持续的距离惩罚。

ES 用确定性评估和群体选择回避了这两个问题；再加上"死亡 −100 远大于累计惩罚"的设计，退化解被排除。

4.4 异构模块的换装

"冻结模块可以拼接"成立，但有条件：

- 同构/同分布：几乎直接可用（论文-45 中同维度 A、B 只需一个小总控）；

- 异构（换了传感器、执行器或表示）：上层需要重新适配，或者加一个对齐层再训练。

本篇的眼睛与翅膀是同一个人设计的固定算法，接口是"大脑的输入层"——它学习怎么读眼睛特征。如果换成训练后冻结的视觉网络，大脑需要重新训练或加对齐层。

五、局限

- 单种子训练；ES 本身有评估噪声（训练中无眼睛组的适应度曾到 800，但最终评测只有 68 步，说明有波动/过拟合训练随机种子）；

- 环境高度简化（二维、圆障碍、固定生成规则）；

- 冻结模块是固定算法，不是训练后冻结的网络；两种来源在本篇中只是概念上等价，没有做对照实验；

- 没有做多代繁殖/交叉、环境变化、多种子统计；

- 指标只有存活步数和被撞率，没有轨迹质量、能耗等。

六、结论

1. **冻结模块 + 可训练小脑**可以在最小规模上学会生存行为：有眼睛时平均存活 317 步，盲飞只有 68 步。

2. **冻结模块可以是固定算法，不必是训练过的网络**；对上层来说，"冻结"的含义是"不可改、只能使用"。

3. **任务设计决定行为质量**：同样的结构，在"追目标"任务里学会躺平或冲锋，在"躲避"任务里才学会左右闪躲。

4. **ES 比 REINFORCE 更稳**：群体选择天然过滤退化解；ES 也是"演化"这一层的直接模拟。

5. 与论文-45 的关系：两者是同一原理的两种形态——冻结组件 + 可训练的适配/决策层；拼什么、怎么拼，取决于异构程度和接口。

附：标注

- **[事实]** 第三节全部存活步数、被撞率、到达率（来自运行结果 JSON）。

- **[解释]** 第四节的机制分析。

- **[推测]** "如果换成训练后冻结的视觉网络，大脑需要重训"——未做对照实验。

- **[不声称]** 不声称这是数字生命的实现；不声称方法新颖；不声称结果可扩展到真实规模。

- 相邻工作：论文-45（冻结子模型拼接）、model stitching、进化策略（OpenAI-ES）。

第二十六篇 · 门：从可塑接口到重配比

单独 DOI：https://doi.org/10.5281/zenodo.24187271

何淇 · 与 DeepSeek（AI）协作

数字生命系列 · 执行器工程（汇总篇，取代论文-49 与工程笔记 03/04/05）

日期：2026-11-13

状态：草稿。数字来自已有实验记录（论文-44/49、工程笔记 01~05 及本次补充）；

标注：[事实] 有记录，[解释] 推演，[推测] 未验证。

一、起点：一个冻结模型，还能改什么

问题不是"怎么微调大模型"，而是：一个已经冻结的系统，如果还要继续用，它能改变的是什么？

拆开看，改变的方式至少三类：加参数（加层、加宽）、改参数（继续训练、LoRA）、以及**不改参数**的前提下，改变已有通路的参与程度。门是第三类：给每条通路一个系数，g=0 不参与，g=1 保持原样，g>1 放大。

所以门最早的定义很窄：**它是组合律上的参与度系数**——调的是"已有的东西用多少"，不是"新增什么"。

二、门是什么：两个面，三档粒度

[解释] 门有两个面：函数面（g 由条件算出来，属于计算）和参数面（g 的数值被保存下来，属于记忆）。同一个门，看哪面决定它是什么。

按粒度分三档：

逐层向量门（任务缩放）：每层一个 d 维向量，所有 token 共用。参数极少，学得快、稳、不易忘；但不随输入变，只能给每层一个全局倾斜。

token 标量门：每个 token 一个标量，只调整体增益。连方向都选不了；初始化不当会直接把模型弄坏（sigmoid 在 0 附近给出 0.5，等于一上来关掉一半通路）。

token 向量门：每个 token 一个 d 维缩放，能按输入选方向。表达力最强，也最容易只在新任务上好、把旧能力挤掉。

三档差别可归到三个坐标：共享范围（层/token）、表达形式（标量/向量）、是否依赖输入。它们不是三个东西，是同一机制的三个角落。

三、门能做什么

[事实] 以下数字来自已有实验。

3.1 换任务：冻结模型 + 门

玩具 6 层 GPT-2（莎翁上训好，冻结）：只训门（768 参数），新任务代码 PPL 1386 → 30 左右；加层（6 层可训练）到 4.27。门大幅适配，但有下限。

真模型 Qwen2.5：

| 模型 | 门参数 | 新领域 PPL | 对照领域 PPL |

|---|---|---|---|

| 0.5B（24 层，d=896） | 21.5k | 91.46 → 32.71 | 22.87 → 24.12 |

| 1.5B（28 层，d=1536） | 43k | 71.26 → 27.32 | 18.69 → 19.07 |

| 3B（36 层，d=2048） | 73.7k | 67.65 → 26.17 | 18.12 → 18.49 |

模型越大：附带损伤越小（5.5% → 2%），增益收窄（27.3 → 26.2）。

3.2 保旧：旧门保底

旧任务门保持旧层 g=1、新层 g=0：旧任务 PPL 4.64，完全不变；新任务门可学到 4.04。两套门各管各的，不遗忘靠"不动它"，不靠训练技巧。

3.3 选择：**5** 样本现场选门

给新任务 5 个带标签样本，每套门算一遍损失选最小：旧任务选旧门、新任务选新门，全对。不需要人写任务描述。

3.4 组合：需要 **A+B** 一起用

| 方案 | 段切换 PPL | token 级 PPL |

|---|---|---|

| 旧门（只用底座） | 77.26 | 270.14 |

| 新门（代码门） | 14.29 | 65.10 |

| 拼门（底座=1 + 代码门） | 17.11 | 70.28 |

| 重训组合门 | **10.49** | **32.69** |

组合任务重训一个组合门最好，拼门不如重训——联合解要整体学。组合门只 1536 参数。

3.5 共享门 + 小偏移

g_A = g_shared + δ_A，g_B = g_shared + δ_B：day1 4.58 / day2 27.27，接近各自专精（4.55 / 29.41）；δ 很小（+0.142 / −0.126）。一套底子加每任务一点方向修正。

3.6 遗忘与回放

会话间只训门：交替任务，累积臂 ≈ 重置臂（门不沉淀）；"会话前衰减"无效；加回放后新/旧任务都保住（6.1~6.4 / 37~43），但单任务峰值下降。遗忘不是把门衰减，是决定"哪些经验值得反复用"。

3.7 生长：卡住就长

固定 6 层 + 门：PPL 28.58（900 步）；卡住加 2 层（8 层）：16.89；一开始就 8 层：7.32。生长能涨，但同样容量下提前给够更好；生长适合算力分批到货/难度未知。

四、门不能做什么

[事实] 门不能加新能力。

合成任务：任务需要 8 维，底座瓶颈 4 维——直接训练 0.3783；冻结+缩放门 0.3783（完全不变）；瓶颈 4→8 为 0.0038（约 100 倍）。

多步组合 y=sin(3·sin(3z))：浅层 0.17754；加门 0.17193（约 -3%）；加层 0.01714（约 -90%）。

同一任务门 vs LoRA：门 0.37802；LoRA（r=16）0.32557（约 -14%）；加宽 0.00383。

门也不能省算力——除非加硬约束。训练后的门变成连续缩放，不做稀疏惩罚/预算，它不会自己保持二值；稀疏性要在训练方式里解决（见第二篇）。

五、定义收拢

把这些合起来，门定成一句话：

> 门是对已有表示通路的重新配比。它改变每个方向被使用的强度，不改变方向的集合。

两个推论：

一是门的极限是"已有能力的可及性"：能调到哪，取决于模型里已经压进了什么。

二是门的种类由三个坐标决定：共享范围、表达形式、是否依赖输入。更好的门应该在三个坐标上找中间点——共享的低秩矩阵门（既有输入依赖，又能加少量新方向，参数量不至于爆炸）。这是推测，没有测。

六、一个负结果：门当诊断没测成

[事实] 我们曾提出：只训门能到多低，可以量出模型里已有多少能力，所以门能当诊断。于是做了一次检验：Qwen2.5-0.5B 冻结，四个领域（笔记/代码/英文/理论），门与 LoRA 各训 150 步，看"门降幅"能否预测"LoRA 收益"。

结果：门降幅 34%~46%，LoRA 降幅 24%~71%；按相对降幅排序，秩相关约 -0.2，不同序——代码的门降幅最小（34%），LoRA 收益却最大（71%）；英文门降幅中等，LoRA 反而最差（24%）。绝对降幅排序一致，但那是基线量级主导的假象。

[解释] 门和 LoRA 调用的能力不同（门是重加权，LoRA 加低秩方向），排序不必一致；英文那组训练样本太少（202 段），LoRA 过拟合。

⇒ "门能当诊断"这个说法，在 4 领域、单种子、150 步的测试下不成立。最多说"除代码外 3 点大致同序"，是弱观察。这条要写进论文，不能只留失败记录在内部。

七、和已有工作的关系

必须承认：我们不是第一个走到这里的。

ReFT / LoReFT：直接对隐藏表示做低秩子空间干预，可按位置（token）选择干预哪些表示；比 LoRA 少 15~65 倍参数，在多类任务上不差或更好。这已经是"逐 token 方向变换"的系统实现。

CeRA：指出 LoRA 有"线性天花板"——秩 16→512，PPL 卡在 3.90 附近，有效秩只用到约 60；原因是线性低秩更新只能旋转、不能弯折空间。加非线性门控后 rank 64 超过 LoRA rank 512，有效秩到 330；同时做了粒度对照（weight-level 优于 module-level）。

我们的工作是这条谱系里最朴素的一档：逐元素缩放，没有低秩、没有路由、没有条件生成。能提供的不是新方法，而是两样东西：

一是受控的阈值测量。合成任务上把"门无效、LoRA 有限、加宽有效"量成数字：底座宽度 2、任务需要 4 维交互时，门/低秩/非线性门控测试 MSE 全部停在 1.262 附近（与不接适配器无差别），只有加单元有效（0.847 → 0.413，随单元数平滑下降）。非线性任务上阈值平滑，"刚好在 k 断崖"不成立。

二是框架位置。已有适配工作回答"哪种方法效果好"；我们把门接到"冻结系统的可塑接口"和"维持成本"上：门是代价最小的一种可塑性，但有结构上限。

八、边界

```

· 合成任务为主；真模型只测了 Qwen2.5 一族

· 门是缩放形式，结论不外推到低秩门/路由门

· 阈值在非线性任务上平滑，不能说"精确在 k 断崖"

· 文献定位只查了 ReFT、CeRA 两支；DoRA、MoE 等没有系统比较

· 门当诊断失败这一条只测了 4 领域、单种子

```

九、一句话

门是重配比，不是加维度；它能调出的是已经压进去的能力，调不出的是没有压进去的能力。这个界限，是我们在合成任务上测出来的，也由已有的适配工作从另一边印证。

第二十七篇 · 门与模型一起训练：门的稀疏性是结构选择

> 单独 DOI：https://doi.org/10.5281/zenodo.24187281

> 何淇 · 与 DeepSeek（AI）协作

> 数字生命系列 · 执行器工程（汇总篇，取代论文-44 的联合训练与执行部分）

> 日期：2026-11-13

> 状态：草稿。数据来自论文-44（DOI 10.5281/zenodo.23858151）及工程笔记 01/02；

> 本篇聚焦"训练方式"与"执行方式"两个变量，不重复 44 的细节。

一、问题

上一篇把门定成"重配比"：门调的是已有通路的参与度，不增加方向，所以它跨不过维度门槛。

但重配比有一个前提没问：**这些通路是什么时候形成的？**

如果门在模型训练的一开始就在，那它参与塑造通路——模型从一开始就知道"有些层可能被跳过"，会把计算安排到别处。如果门是模型冻结之后才加的，那它只能在一个已经定型的结构上做取舍，而那些层在预训练时都被"用惯了"。

同样的门，同样的参数，训练方式不同，它能做到的事可能完全不同。这篇只测这个变量，再补上执行侧（怎么算才算省）。

二、设置

门的形式：每层一个内容门 σ(W·h+b)，每个 token 每层各自决策。训练时用拉格朗日乘子控制平均活跃层数：损失 = 语言建模损失 + λ·平均活跃层数，λ 自适应调整。不人工指定哪层该关。

两种执行方式：

- 保留 K/V 的稀疏执行：K/V 全算，门关的 token 跳过 Q/注意力输出/MLP；

- 容量路由（MoD 风格）：每层固定容量 K，按门分数取 top-K 个 token。

对照：从零训练（1.2M 字符模型、GPT-2 124M、Qwen2.5-1.5B）vs 预训练 + LoRA 后训练（Qwen2.5-3B/7B）。数据为 tinyshakespeare（1MB）或 TinyStories（20MB/100MB），评测为验证集 PPL。

三、结果：训练方式决定门能做到什么

| 设置 | 平均用层 | 代价（PPL） |

|---|---|---|

| 1.2M 字符模型，从零 | 2.97/6（50%） | ≈0 |

| GPT-2 124M，从零，1MB | 5.30/12（44%） | +6.8% |

| GPT-2 124M，从零，100MB | 5.81/12（48%） | +2.7% |

| Qwen2.5-1.5B，从零，100MB | 12.32/28（44%） | +1.1% |

| Qwen2.5-3B，预训练+LoRA | 32.79/36（91%） | +18% |

| Qwen2.5-7B，预训练+LoRA | 25.70/28（92%） | +38% |

| Qwen2.5-7B，目标 24 层 | 24.01/28（86%） | +85% |

| Qwen2.5-7B，目标 22 层 | 22.27/28（80%） | +82% |

三个观察：

一是从零训练时，门可以便宜到几乎免费：1.2M 省一半层数不损质量；1.5B 省 56% 层数只多 1.1%。

二是从零训练时，门自己会做结构选择。1.5B 上最后 9 层（L19~L27）几乎全部关闭，前面保留大部分层；GPT-2 上是第 3 层几乎不用、末两层回到 0.74~0.75。这不是我们指定的，是训练自己找到的。

三是冻结 + 轻量后训练时，门压不动。3B/7B 的层数只压到 91%~92%，代价 +18%~+38%；想压到 86%/80%，代价跳到 +85%/82%，7B 那次还发生了门控塌缩（PPL 升到 1243，层数掉到 20）。这不是"大模型不能压缩"的结论——只能说明这种后训练方式不行。

四、门本身不伤质量，伤在"怎么执行"

在 GPT-2 门控模型（7.48/12 层，PPL 12.03）上：

- 只算门开 token、K/V 全留（open_only）：PPL 12.032，与稠密门控 12.033 精确等价；

- 容量路由（门开优先 + 静态 K_max，factor=1.00）：名义省 37.9% 计算，PPL 12.107（+0.6%）；

- 质量拐点在 K_max ≈ 平均门开数（factor 1.0~1.05）；factor=0.95 时 PPL 14.458。

这说明门控本身不带来质量损失；差异来自执行方式。代价出现在更强的地方：注意力子集化（跳过 token 不产生 K/V）在已训练模型上 +6.8%（quantized）、half +16.6%、quarter +55%；从零训练两次都崩（PPL 350.99 / 393.71）。这不是"子集化不可行"的结论——MoD 原方法用独立路由器 + 容量路由，在更大规模上验证过这个语义；我们的路由方式不够。

[事实] 真实墙钟（重要负结果）：自写 gather/索引实现下，batch 32 为 0.89~1.22x，batch 64 为 1.01x；早期一次 1.22x 与后期 0.89x 的差异说明结果高度依赖实现细节。我们测到的是名义计算量，真实加速是独立的工程问题。

分支独立门控失败：每层两个门（注意力、MLP 分开），从零训练 12000 步 final PPL 14.484；联合门控 4000 步就到 12.033。分支拆分评测：MLP 压到 25% → PPL 558.95；注意力压到 25% → 69.45。初步图景：MLP（权重承载的变换）与 K/V（记忆）是承重部分，Q 侧更新相对可省——单模型观察，不能当结论。

五、解释与边界

[解释] 从零训练时，全网在"可能跳层"的约束下共同学习，可以把计算集中到少数层；冻结模型没有这个机会：每一层都被预训练用惯了，LoRA 的低秩更新改不动这个结构。所以门的稀疏性更像一个**结

构选择**，而不是事后能强加的适配。

边界：单种子为主；数据 1MB~100MB；仅 PPL；从零训练组欠训练（8000 步约 0.33 epoch），绝对数不代表真实能力；未与 MoD 原文做同设置对比；未测 1B 以上从零训练；墙钟结果受实现影响大。

六、与已有工作的关系

门控、稀疏执行、容量路由、早退同属条件计算；MoD 已报告过容量路由的可行性，LayerDrop 等也做过训练时的层丢弃。我们不声称提出新方法。

我们的观察是：**同一个门，从零训练与冻结后加，能做到的事差一个量级**；而且从零训练时门会自己关闭整段末端层，这像是把"哪些层是承重的"学了出来。这个对照在已有文献里不常被单独强调，但它是一个工程事实，不是方法贡献。

七、和第一篇的关系

第一篇说门是重配比，跨不过维度门槛；这篇补上前提：重配比发生在什么结构上，取决于门是否参与训练。冻结时，门只能在定型结构里选；共同训练时，门参与塑造那个结构，于是它能"关掉一半层"而不伤功能。

一句话：**门在冻结模型里是开关，在共同训练里是结构选择。**

第二十八篇 · 上下文卸载与局部适配：一个多模型拼接 + 独立记忆通道的思路（思路草稿）

> 单独 DOI：https://doi.org/10.5281/zenodo.24218262

> 何淇 · 与 DeepSeek（AI）协作

> 数字生命系列 · 论文-52（v1，思路草稿）

> 日期：2026-11-14

> 状态：思路草稿。提出方向 + tiny/合成初步实验，结论为方向性证据；后续可能被修正。

摘要

我们提出一个有待验证的工程思路：不让单一模型承担全部上下文与全部能力，而是把系统拆成"冻结的判断核心（一个或多个冗余模型）+ 独立的记忆通道 + 顶层的组合/仲裁模型"，适配器只作用于局部，组合权重固定或受限。目的不是让模型更聪明，而是让"加新能力"这件事尽量不破坏旧能力。

我们用 tiny 模型和合成任务做了三组初步实验，全部为小规模、单种子，结论只作为方向证据，不作为性能声明。

一、问题

1. 单模型的上下文窗口有限，扩窗口的代价随长度快速上涨。

2. 把新信息写进权重（微调、LoRA、记忆固化）会改变表示空间；表示空间一改，旧判断跟着变——我们前面的实验里，门/适配器可以把旧任务 PPL 从 1.74 推到 11。

3. 因此"学新东西"和"保住旧判断"在单模型内部是冲突的。

我们想问的是：**能不能通过系统结构，把"新"放在局部，让旧的部分逐位不变？**

二、提议的结构

```plain
输入 ──┬──→ 模型 A（冻结，判断核心）
       ├──→ 模型 B（冻结，判断核心，可与 A 相同或不同）
       ├──→ 记忆通道（检索式 / 压缩式 / 状态式；不修改 A、B）
       │     └──→ 少量记忆 tokens
       └──→ 顶层模型 C（组合与仲裁；可训练，固定后冻结）
```

三条设计约束：

1. **判断核心冻结**：A、B 的表示空间不动，它们的判断逐位保持；

2. **记忆走独立通道**：记忆的增删只改变 C 的输入，不改变任何模型的权重；

3. **组合权重固定**：C 学到组合方式后锁死，避免"适配目标时整体漂移"。

[推测] 如果三条都满足，旧输入上的输出应该完全不变；新信息只通过 C 的输入通道进入。

三、初步实验

3.1 顶层判断对底层漂移的敏感性（合成任务，秒级）

结构：A、B 完全相同（判断 sum(x)>0），C 在 [logits_A, logits_B] 上训练；只对 B 微调，使它慢慢转向另一个规则（x[0]>0）。

| 漂移步数 | B 在原任务 | C 准确率 | C 判断翻转率 | A/B 表示重合度 CKA |

|---|---|---|---|---|

| 0 | 0.988 | 0.989 | 0 | 1.000 |

| 25 | 0.924 | 0.983 | 0.01 | 0.952 |

| 50 | 0.786 | 0.971 | 0.026 | 0.771 |

| 100 | 0.554 | 0.941 | 0.056 | 0.547 |

| 200 | 0.575 | 0.895 | 0.104 | 0.375 |

| 400 | 0.575 | 0.832 | 0.167 | 0.226 |

| 800 | 0.577 | 0.759 | 0.240 | 0.166 |

| 1200 | 0.578 | 0.709 | 0.290 | 0.151 |

观察：

- C 的判断不是突变，而是随 B 漂移**连续退化**；B 已掉到随机水平时，C 仍保持 0.94（因为 A 还在，顶层可以靠 A 撑）；

- A/B 的表示重合度（CKA）单调下降，与 C 的准确率走势一致，可作为"顶层风险"的预警指标。

[边界] 合成任务、线性顶层、单种子；漂移方向是"另一个规则"而非随机噪声。换非线性顶层或对抗性漂移，可能出现阈值效应。

3.2 独立记忆通道（合成任务，秒级）

结构：键 0..99 → 值 0..9；A、B 只训练过前 50 个键；C 输入 = [A 输出, B 输出, 记忆 one-hot, 记忆已知标志]。

| 新键上已填充记忆数 | C（有记忆通道） | C（无记忆通道） |

|---|---|---|

| 0 | 0.126 | 0.130 |

| 10 | 0.308 | 0.130 |

| 20 | 0.487 | 0.130 |

| 30 | 0.665 | 0.130 |

| 40 | 0.836 | 0.130 |

| 50 | 1.000 | 0.130 |

同一过程中 A、B 在新键上的准确率保持约 0.08，未被改动。

观察：独立记忆通道确实能让顶层判断"从外部学到新事实"，且不改变 A/B 的权重与判断。

3.3 盲信与仲裁（合成任务）

在上述结构下，把测试键的记忆故意写错：

| 模型 | 测试区 | 跟随错误记忆 | 判对率 |

|---|---|---|---|

| C_clean（训练时记忆从没错） | 旧键（A/B 会） | 99.8% | 0.3% |

| C_clean | 新键（A/B 不会） | 100% | 0% |

| C_noisy（训练时 20% 记忆写错） | 旧键 | 55.5% | 41.6% |

| C_noisy | 新键 | 94.9% | 0.6% |

进一步用"现实混合"训练（80% 冗余且记忆对、10% 冲突、10% 新键），并加入 A/B 置信度输入：

| 测试 | 无置信度输入 | 有置信度输入 |

|---|---|---|

| 旧键，记忆对 | 0.987 | 0.986 |

| 旧键，记忆错（A/B 对） | 0.565（跟随错记忆 37%） | 0.555（37%） |

| 新键，记忆对 | 0.831 | 0.818 |

| 新键，记忆错 | 跟随 80% | 跟随 78% |

观察：

- 盲信是**训练分布**造成的：只要训练时记忆永远正确，C 会无条件信记忆，哪怕 A/B 明明知道；

- 训练中混入冲突能显著削弱盲信（跟随错误记忆 99.8% → 37%），但**没有完全学会以 A/B 为准**（理想应接近 99% 判对）；

- 显式加"置信度"输入没有带来额外收益，因为 logits 里已经包含该信息。

3.4 两个专家的并联与"只改一个成员"（**tiny char** 模型）

两个 tiny char 模型分别只在 A（代码）和 B（笔记）上训练，C 是它们在 C 文本（英文）上的组合/适配对象。

- 单模型：A 自测 PPL 1.74（in-sample），B 自测 23.36；A 测英文 381.74，B 测英文 23.72；

- 固定权重组合（w 为 A 的权重）：w=0.2 时 A 5.52 / B 26.24 / 英文 23.21；w 越大，A 越好、B 与英文越差；

- 适应英文（只给一个成员加门 + 学 w）：无门对照 22.80；门在 A 19.17；门在 B 17.30；但 A/B 都变差（A 8.03→10.7~11.1，B 24.48→29~30）；

- 单模型 A + 门：英文 39.59（相对 381 大幅改善），但 A 从 1.74 掉到 11.12。

观察：组合能带来有限互补；"只改一个成员"并不能保住其他成员的表现，只要组合权重可学，它就会跟着适配目标一起漂。

[边界] tiny char 模型、A/B 的评测为 in-sample（偏乐观）、单种子；英文一词在此处指 english-essentials.md。

3.5 仲裁实验：先验决定取舍（合成任务）

在 3.2/3.3 的设定上继续加"仲裁"机制，共三版：

- 第 1 步：给 C 加"分歧特征"（记忆与 A/B 的 JS 距离、A/B 自身分歧），冲突样本比例提高；

- 第 2 步：拆出显式裁判 R，输入同特征，输出 trust，最终判断 = trust·记忆 + (1−trust)·A/B；

- 第 3 步：硬规则基线（记忆未知→信 A/B；A/B 不确定→信记忆；A/B 确信且分歧大→信 A/B），以及把裁判训练时记忆正确率改为 90%。

| 测试 | 第 1 步 | 第 2 步（50/50 先验） | 第 3 步（90% 先验） | 硬规则 |

|---|---|---|---|---|

| 旧键冲突：判对 | 0.663 | 0.716 | 0.321 | 0.763 |

| 旧键冲突：跟随错误记忆 | 24% | 21% | 67% | 19% |

| 新键给对记忆：采纳 | 0.743 | 0.566 | 0.969 | 0.604 |

| 新键写错记忆：跟随 | 69% | 53% | 94% | 61% |

| 旧键正确：判对 | 0.984 | 0.973 | 0.994 | 0.942 |

观察：

- **仲裁不是"学会就好"，而是被先验决定**：训练中记忆正确率 90% 时，裁判学会"多信记忆"——新事实采纳率升到 0.97，但冲突判对率崩到 0.32；反过来 50/50 先验时冲突判对 0.72、新事实只采纳 0.57。

- 第 1 步的分歧特征有效（冲突判对 0.565→0.663），但只是把折中位置挪了一点。

- 硬规则给出另一个折中点（冲突 0.76 / 新事实 0.60），并不比训练版全面差。

- 原因：没有外部信号时，"该不该信记忆"在原理上不可判定，只能依赖先验；训练分布就是先验。

[边界] 合成任务、单种子、错误类型是随机换值；"先验"为人为设定的训练比例。真实场景需要记忆来源/可靠性元数据或验证通道，才能打破这个取舍。

四、谨慎解读

[事实] 上述三组实验支持三点：

1. 顶层判断对底层漂移**敏感且连续**，表示重合度可作预警；

2. 独立记忆通道能在**不改判断核心**的前提下注入新信息；

3. 但记忆通道默认会**盲信**，仲裁需要训练；且仲裁结果被"记忆可靠性先验"决定——先验高则采纳新事实强、识别错误弱，反之亦然。要同时做到两者，需要记忆来源/可靠性元数据或外部验证信号；

4. "只改一部分成员"不等于"旧能力不动"：只要组合权重可学，它就会跟着适配目标漂。

[不声称]

- 不声称本结构优于现有方法（检索增强、MoE、stacked generalization、model stitching、adapter routing 都有近亲）；

- 不声称实验结论可外推到真实大模型（全部为 tiny/合成、单种子）；

- 不声称"记忆通道能替代长上下文"——它是有损、按需的扩展，瓶颈在检索与仲裁。

[已知负结果]

- KV 压缩三版协议失败（跨文档无信号、同文档无信号、针测试非单调）；

- 逐段秩-1 向量门 12/12 选择恒等，无收益（且可能是门训练不足）；

- 连续学习 tiny 实验因任务不可学而无效；

- 一个早期实验因注意力缺 causal mask 产生"抄下一个字符"的假 PPL（≈1.05），已定位并修正。

五、可检验的下一步

1. **仲裁训练**：用更高冲突比例 + 结构化"不一致检测"模块，看能否把旧键冲突判对率从 56% 推到接近 100%；

2. **重组权重的影响**：锁死 w 后重复 3.4，"只改一个成员"能否真正保住 A/B；

3. **压缩 vs 检索 vs 状态**三种记忆通道的对比（各自的信息保留率、更新成本、仲裁难度）；

4. **漂移预警**：CKA 下降多少时顶层判断开始不可接受，给出阈值曲线；

5. **规模验证**：把同一结构搬到 0.5B/1.5B 冻结模型上，检验结论是否保持。

六、定位

这是一份**工程思路草稿**：提出一个"判断核心冻结 + 记忆独立通道 + 顶层仲裁"的结构，并用 tiny/合成实验给出方向性证据。它不是理论，也不是性能声明；它最大的价值可能在于三条负面提醒：

1. 改权重一定改判断；

2. 独立记忆一定带来盲信；

3. 只改一部分，不等于不漂。

结构能做的，是把这些问题变成可测、可约束的量。

第二十九篇 · 工程笔记 **01**：门控与模块化的失败、修正与判断（持续更新）

> 何淇 · 与 DeepSeek 协作

> 数字生命系列 · 工程笔记（新板块第 01 篇）

> 日期：2026-11-03

> 定位：过程记录，不是结果论文。写我们怎么想、哪里错了、修正成什么、还剩什么没搞清。以后按 02、03…… 继续加。

一、这篇笔记是什么

正式论文只写"我们做成了什么"，工程笔记写"我们是怎么走到那一步的"。

它记录三类东西：

- **最初的设想**：我们以为门是什么、想用它做什么；

- **失败的实验**：哪些方向试过、怎么失败、失败给出什么信息；

- **形成的判断**：从失败和对照里沉淀下来的经验，以及哪些判断还只是初步的。

写它的原因很简单：失败里包含的信息，往往比成功的数字更有用；而且如果不写下来，过几个月就只剩结论，过程没了。

二、最初的设想

最早我们想的门不是"省算力"，而是**判定**：

```

· 在框架里，时间是算子序列：s_{t+1} = I(s_t)

· 判定 J 决定：这一步做什么、什么时候停

· 门控 = 判定的工程实现：

· 门开 → 这一步算

· 门关 → 这一步跳过

⇒ 门是"给每个 token 分配步数/时间"的东西

```

所以第一版的门很小：每层一个 `Linear(d, 1)`，`g = sigmoid(W·h)`，乘在该层的残差贡献上。它不决定输出什么字，只决定这一层对这个 token 算多少。

三、失败的实验

① 注意力子集化（跳过 token 不产生 K/V）

在已训练的门控模型上直接切换语义：PPL 12.033 → 12.855（+6.8%）。从零训练两次尝试都崩：350.99（活跃率塌到 8.3%）、393.71（容量没塌也崩）。

结论：在我们的门控路由上，"被跳过的 token 从注意力里消失"代价明显。但不推广为"子集化本身不可行"——MoD 原方法用独立路由器 + 容量路由，在更大规模验证过。

② 分支独立门控（注意力门、MLP 门分开）

从零训练 12000 步：PPL 14.484；联合门控 4000 步就到 12.033。分支拆分不是更好，而是更差。分支拆分评测还显示：MLP 压到 25% → PPL 558.95；注意力压到 25% → 69.45。

结论：MLP（权重承载的变换）与 K/V（记忆）是承重部分，Q 侧更新相对可省。

③ 预训练 3B/7B + LoRA 后训练

层压不动（80%~92%），PPL 反升（+18%~+85%），7B 还出现门控塌缩（PPL 1243）。

结论：这条只说明"预训练 + 轻量后训练 + 小数据"不行，不说明"大模型不能压"。要回答后者需要从零训练大模型，成本是主要障碍。

④ 追目标任务里"有眼睛反而不如盲飞"

同一套冻结眼 + 冻结翅膀 + 可训练大脑：

| 配置 | 到达率 | 撞障碍 | 撞墙 | 超时 |

|---|---|---|---|---|

| 有眼睛 | 33% | 8.7% | 7% | 51% |

| 无眼睛 | 80.7% | 19.3% | 0% | 0% |

有眼睛的个体看到障碍后变保守，大量超时；没眼睛的径直冲目标，反而到达率高。

结论：任务设计比算法更关键。眼睛提供"危险信号"，但不提供"绕过去"的梯度；换成"躲避"任务后，眼睛的信息才直接变成行为。

⑤ 近距离惩罚让策略学会"躺平/自杀"

在"追目标 + 避障"任务里加"离障碍越近罚越重"后：完整版到达率 0%、超时 99.7%（学会了原地不动）；另一版学会快速结束回合以躲避持续惩罚。

结论：惩罚设计要防止退化解；死亡代价必须远大于累计惩罚，且不能给"停下来"任何好处。

四、修正

从上面这些失败里，我们修正了几件事：

```

· 保留 K/V 的稀疏执行与稠密门控精确等价（12.032 vs 12.033）

⇒ 门控本身不伤质量；质量差异来自执行方式

· 软门是本体，硬跳是省算力的"硬化"

⇒ 训练用软门，部署再硬化

· 判断"能不能压"的关键变量是训练方式，不是参数量

⇒ 从零训练 1.2M~1.5B 都能用约一半层；预训练+LoRA 压不动

· 冻结模块 + 可训练接口是同一原理的多种形态

⇒ 论文-45 的拼接、论文-46 的数字蚊子都在这条线上

· 任务设计优先于算法选择

⇒ 同样的结构，换个任务就从"躺平"变成"会躲"

```

四之补、约束方式的差别：从 **LAMBDA** 到拉格朗日目标

还有一个容易被忽略的差别：**同一个门，被不同的约束训练，行为完全不同。**

最早那版（论文-38 风格）用的是一项目标正则：

```

loss = 语言建模损失 + LAMBDA × 门权重的平均值

```

它只是"鼓励门变小"，没有"平均只能开几层"的目标。结果是：门大量保持开着（开着能降损失，那点正则压不住），主要的收益是适配（PPL 大幅下降），稀疏性没有保证。

后面从零训练时换成了拉格朗日约束：

```

· λ 自适应调整

· 目标：平均开层数 = k（比如 6 层）

· 不达标就加大 λ，超标就减小

⇒ 门必须在"降损失"和"少开层"之间取舍

```

差别总结：

| | 最早的 LAMBDA | 后面的拉格朗日目标 |

|---|---|---|

| 约束对象 | 门权重大小 | 平均开层数 |

| 是否可控 | 不可控（λ 固定，效果不确定） | 可控（按目标调） |

| 目的 | 让门别太大 | 真的压层数 |

| 结果 | 主要适配，稀疏性弱 | 稀疏、可测、可扫曲线 |

所以"门学到了什么"取决于被什么约束：没有目标约束时，它是软适配器；有目标约束时，它才是可调预算的稀疏开关。

五、今天的对照：**sigmoid** 门 **vs** 纯线性门

为了确认"门要不要压缩（sigmoid）"，我们做了一组最小对照：

```

· 冻结 GPT-2 small（不更新）

· 只训练 12 个门（每层 Linear(768→1)，约 9k 参数）
```

· 数据：莎士比亚 1MB，训练 600 步

```

结果：

| 组 | 验证 PPL |

|---|---|

| 无门（冻结基线） | 44.31 |

| **sigmoid 门** | **24.59** |

| 纯线性门（无 sigmoid） | 27.48 |

门值分布：

```

· sigmoid 门：全部在 0~1，层间分化明显，稳定

· 纯线性门：出现负值（最低 -27.3）和极大值（最高 17.5）

· 层 11 方差 2.5，训练不稳定

⇒ 无界放大/取负没有带来收益，反而更差

```

结论：门需要压缩到有界区间（sigmoid）；纯线性门不是"更自由"，而是"更不稳定"。

## 六、形成的判断（当前版本）

```

① 门是"每层一个、按 token 内容算开度"的开关

· 不是每 token 一个门参数；参数按层共享，开度按 token 算

② 门的三重身份：

· 时间/步数分配器（最初动机）

· 稀疏开关（省算力）

· 轻量适配器（冻结模型 + 9k 参数，PPL 44→25）

③ 软混合是本体，硬跳是硬化；不考虑算力时应停在软门

④ K/V 必须保留；MLP 是承重墙

⑤ 关键变量是训练方式，不是参数量

⑥ 数据越多，"能压的层"可能越少（初步观察，未定）

⑦ 冻结模块 + 可训练接口可以组合，但接口要对齐

⑧ 任务设计比算法选择更决定行为质量

```

## 七、还没搞清

```

· 数据量曲线：压缩比是收敛到渐近线，还是持续变难？

· 迭代收敛：冻结前端 + 可训练循环块，能否用迭代换精度？

· 模块跨代演化：先天回路本身怎么"演化出来"？

· 门作为解释：门模式能否稳定地解释"模型为什么这样答"？

· 异构模块接口：换传感器/执行器后，上层要重训多少？

· 大模型从零 + 门控：3B/7B 从零训练是否能压？成本是障碍

```

## 八、更新方式

以后每有什么值得记的过程、失败对照、判断修正，就在这里加一篇：

```

· 工程笔记 02：……

· 工程笔记 03：……

```

不追完整，只追"当时怎么想、后来怎么改"。这样这条工程线的过程不会丢。

> 一句话：正式论文写"结果"，工程笔记写"我们怎么从错的地方走到对的地方"。

## 第三十篇 · 方向观察 **01**：加层、回放与蒸馏——大模型睡眠巩固（待复核）

> 单独 DOI：https://doi.org/10.5281/zenodo.23927099

> 何淇 · 与 DeepSeek 协作

> 数字生命系列 · 执行器工程 / 方向观察（内部编号第 47 篇）

> 日期：2026-11-04

> 状态：方向观察与最小实验——极小模型、字符级、单种子；只验证机制，不作方法宣称。

### 摘要

一个训练完成的模型，如何在不遗忘旧能力的前提下继续学新东西？本篇记录一组最小实验，并给出两个应用方向的判断。

基础是一个 1.21M 参数的字符级 6 层模型：day1 数据为莎士比亚，day2 为 Python 源码。主要观察：

1. **加层能学新能力，且比全参微调少遗忘**：全参微调 day2 时 day1 PPL 从 4.64 崩到 46.3；冻结基础加 6 层后 day1 为 18.2、day2 为 4.26；加层并回放 day1 后 day1 保到 4.83、day2 为 6.60。

2. **门有三个作用**：轻量适配（冻结模型只训门，PPL 44.31→24.59）、步骤分配（每 token 每层决定算不算）、模式切换（单模式回放时零和；双模式回放可两边都保）。

3. **极小回放可以接近全量回放**：100 条旧样本、10% 混合，day1 9.05、day2 5.02（对照：不回放 day1 18.2；全量回放 day1 4.78、day2 8.67）。

4. **蒸馏压缩可以打包**：教师（12 层 gated，已巩固）day1 6.96/day2 4.50，学生（8 层更宽、无门）day1 7.42/day2 2.18。

据此给出两个方向（都只到"可行"级别）：个人 AI 的睡眠巩固；大模型新语料的增量更新。并明确：实验是玩具规模，不声称优于现有做法。
```

一、问题

模型训练完、部署后，知识停在训练那一刻。要更新，有几条路：

- **外部记忆**：检索、上下文。便宜、无损，但不改变模型本身；

- **全参微调/继续训练**：便宜一些，但会灾难性遗忘（我们实测 day1 PPL 从 4.64 到 46.3）；

- **完全重训**：质量最好，但要全部旧数据 + 新数据，成本最高；

- **冻结旧模型 + 加层**：旧模型只做推理，新层学新东西。

本篇关心最后一条：加层能否承载新能力、如何防止干扰旧能力、如何把生长打包。

二、预备：模型、加层与门

2.1 基础模型

```

· 字符级语言模型：6 层 Transformer

· 宽度 d=128，4 个注意力头，MLP 隐层 512

· 参数量 1.21M

· 在 day1（莎士比亚）上从零训练到收敛

· 训练后冻结：参数不再更新

```

用这么小的模型，是为了让所有实验在 CPU 上几分钟跑完，把机制问题与规模问题分开看。

2.2"加层**"**是什么

```

· 基础模型 6 层，冻结

· 在它上面再接 6 层新的 Transformer

· 新层随机初始化
```

· 输入输出维度相同（128）

· 共享冻结的嵌入层、位置编码、最终 LayerNorm 和输出头

⇒ 总模型 12 层：

前 6 层 = 冻结的旧模型

后 6 层 = 新加的可训练层

```

**2.3"门"是什么**

门是每层一个很小的模块，决定这一层对当前 token 起多大作用：

```

· 每层一个线性层：Linear(d → 1)，d=128

· p = sigmoid(W·h + b)，一个 0~1 的数（h 是该层输入）

· 这一层前向变成：

h_out = h + p · (Block(h) − h)

· p≈1：这一层完整计算

· p≈0：这一层近似跳过

· 中间：按比例混合

⇒ 门不产生输出，只控制"这一层算多少"

```

门的位置逐实验说明：

- **后 6 层门**：只给新加 6 层加门，前 6 层不加（等价 p≡1）；

- **12 层全加门**：前 6 层（冻结权重）和后 6 层都加门，前 6 层的门也可训练。

门初始化：使开率起始约 0.5~0.9（接近开）。

**2.4** 训练流程
```

```

阶段 1（day1）：训练基础 6 层（无门）

阶段 2（day2）：冻结基础，只训新 6 层（可带门）

阶段 3（巩固）：混合 day1/day2 训练（可只训门）

阶段 4（蒸馏）：用混合数据训练一个稠密学生

```

三、门的三个作用

3.1 轻量适配

```

· 冻结模型，只训练门（论文-38：12 个门约 9k 参数）

· GPT-2 small 冻结 + 12 个门：PPL 103.1 → 64.1

· 我们的小实验：44.31 → 24.59

⇒ 门像一个极小的适配器，把冻结模型的层贡献重新加权

· 但它不能学新表示，只能重新组合已有层

```

3.2 步骤分配

```

· 每个 token、每一层：算还是不算

· 从零训练时门开率会学出结构：

· 第 0 层几乎全开

· 中高层对部分 token 关闭

· 我们 1.5B 从零训练的模型：最后 9 层几乎全关
```

⇒ 门在分配"这个 token 走几步"

```

**3.3** 模式切换

```

· 不同数据/模式之间切换层的开合

· 单模式回放：零和

· 只回放 day1：day1 恢复 4.72，day2 崩到 1415

· 双模式回放：两边都保

· day1 6.96（从 26.70 恢复），day2 4.50

⇒ 门能做切换，但要"两种模式都见过"才不零和

```

## 四、最小实验

**4.1** 单任务上加 **6** 层

在 day1 上训练 6 层基础模型，然后加 6 层：

| 组 | 配置 | 验证 PPL |

|---|---|---|

| A | 基础 6 层 | 4.34 |

| B | 冻结 6 + 后 6 层可训练（门只在后 6 层） | 4.16 |

| C | 冻结 6 + 后 6 层可训练（12 层全加门） | 4.17 |

| D | 冻结 6 + 后 6 层可训练（不加门） | 4.20 |

观察：加层带来约 3% 提升；门与无门差别在噪声内；新层门开率只有 0.16~0.5，说明额外深度用处不大。

**4.2** 持续学习（**day1 → day2**）
```

| 组 | 做法 | day1 PPL | day2 PPL |

|---|---|---|---|

| A | 基础（只见过 day1） | 4.64 | 1386.7 |

| B | 全参微调 day2 | 46.3 | 1.52 |

| C | 冻结 + 加 6 层（无门、无回放） | 18.2 | 4.26 |

| D | 冻结 + 加 6 层 + 回放 day1（各 50%） | 4.78 | 8.67 |

| L | 长期学（新层 + 12 层门，day2） | 26.3 | 3.76 |

| LR | 长期 + 回放 | 4.83 | 6.60 |

观察：全参微调 day2 最好但 day1 崩；加层遗忘更轻；回放让 day1 基本保住、day2 略差。

4.3 门作为切换器

在 L（长期学完）基础上冻结所有层，只训门：

| 组 | 做法 | day1 PPL | day2 PPL |

|---|---|---|---|

| REC | 只训门 + 回放 day1 | 4.72 | 1415.6 |

| D2 | 只训门 + 回放 day2 | 24.11 | 3.79 |

| DUAL | 只训门 + 双模式回放 | 6.96 | 4.50 |

| DUAL6 | 门只在后 6 层的双模式回放 | 6.11 | 4.48 |

观察：单模式切换零和；双模式回放能让门同时保住两边；门放后 6 层与 12 层差别很小。

4.4 极小回放 **vs** 极小约束

| 组 | 做法 | day1 PPL | day2 PPL |

|---|---|---|---|

| C | 不回放 | 18.2 | 4.26 |

| D | 全量回放（50%） | 4.78 | 8.67 |

| E2 | 100 条旧样本、混合 10% | 9.05 | 5.02 |

| E3 | 100 条旧样本 KL 约束（权重 1.0） | 4.70 | 29.74 |

观察：100 条样本就能显著缓解遗忘；极小回放和全量回放各有优劣（旧能力 vs 新能力）；KL 约束权重 1.0 太强。

4.5 蒸馏压缩

教师 = 12 层 gated（基础 + 6 新层，双模式回放巩固）：

| 模型 | day1 PPL | day2 PPL | 参数量 |

|---|---|---|---|

| 教师（12 层 gated） | 6.96 | 4.50 | ~2.5M |

| 学生 A（8 层，d=160，无门） | 7.42 | 2.18 | 2.68M |

| 学生 B（12 层，d=128，无门） | 7.44 | 2.83 | 2.55M |

观察：学生保留两边能力（day1 差约 6%、day2 更好），门被完全去掉。

五、两个方向

5.1 个人 **AI** 的睡眠巩固

```

· 每人一个"个人层"（新加层 + 门），共享底座

· 白天：上下文/记忆保存（短期、无损）

· 晚上：把当天语境训进个人层，只训个人层和门

· 少量回放共享数据防遗忘

· 定期蒸馏压缩个人层

· 好处：隔离、隐私（数据只进个人层）、可回滚

· 可行性：我们的加层+回放+蒸馏实验支持机制可行；
```

真实规模未验证

```

**5.2** 大模型新语料的增量更新

```

· 旧模型冻结，只做推理

· 新语料到达 → 在其上训新层

· 可连续更新，不必全量重训

· 层积累后定期蒸馏打包

· 这是"全量重训"的增量替代，不是免费午餐

· 适用边界：

· 领域扩展、同类型新语料：可行

· 需要旧模型从未具备的低层特征（全新语言/模态）：不够，

可能要改嵌入或重训底层

· 代价：前向变贵（多出的层）、蒸馏一次成本

```

两个方向都只到"可行"级别；我们的小实验只验证机制，不验证规模与真实收益。

## 六、与 **Google sleep framework** 的对照

Google 2026 年的睡眠框架（arXiv:2606.03979，*Language Models Need Sleep: Learning to Self-Modify and Consolidate Memories*）把运行分成清醒与睡眠两阶段：

- **Memory Consolidation**："Knowledge Seeding"——把较小自身的记忆向上蒸馏进更大的网络，在扩容同时保住知识；

- **Dreaming**：用 RL 生成合成数据课程，排练新知识、精炼旧能力。

| | Google sleep | 本篇 |
```

|---|---|---|

| 扩容方式 | 蒸馏进更大网络 | 冻结旧模型 + 加 6 层 |

| 防遗忘 | 向上蒸馏 + 回放 | 双模式回放（教门两种模式） |

| 巩固信号 | RL 生成合成数据 | day1/day2 混合数据 |

| 切换机制 | 无显式门 | 门（单模式零和） |

方向一致：都把"短期记忆 → 长期参数"当核心，都靠回放防遗忘。差异在机制。

七、方向判断与局限

判断（只在玩具规模上成立）

```

· 加层可以承载新能力，遗忘比全参微调轻

· 门是"细调旋钮"与"切换器"，不是能力开关

· 回放是关键；极小回放（100 条）就有明显效果

· 蒸馏可以把生长打包成固定大小、无门的模型

· "生长 → 巩固 → 打包"构成一个自洽循环

```

局限

```

· 极小模型（1.21M）、字符级、单种子

· day1/day2 分布差异极大（莎士比亚 vs 代码）

· 学生用硬标签混合数据训练，未用教师软标签或 RL 课程

· 没有在真实规模模型上验证

· "门放后 6 层略好"的差异在噪声内，不能当结论
```

```

**结论**

```

· 加层 + 回放 + 蒸馏是一条可行的持续学习循环

· 它的两个可能应用方向：个人 AI 的巩固、大模型的增量更新

· 两者都只到"可行"级别；真实价值需要真实规模验证

```

附：标注

- **[事实]** 第四、五节全部 PPL 数字（本地运行结果 JSON）。

- **[方向判断]** 第五节两个方向、第七节判断。

- **[不声称]** 不声称方法新颖；不声称可扩展到真实大模型；不声称优于全参微调/外部记忆。

- 相关：Google sleep framework [arXiv:2606.03979](https://arxiv.org/abs/2606.03979)；

[Google Research 博客](https://research.google/blog/the-sleep-framework-a-new-way-for-llms-to-retain-knowledge-in-continual-learning/)。

- 脚本：`continual_add.py`、`continual_time.py`、`continual_gate_dual.py`、`continual_gate_dual6.py`、`small_replay.py`、`distill_compress.py`、`gated_extend.py`、`gated_extend_nogate.py`；结果 JSON 同目录。

## 第三十一篇 · 工程笔记 **02**：一个自学习的 **LLM** 模型——冻结、成长与切换

> 单独 DOI：https://doi.org/10.5281/zenodo.24046399

——冻结、成长与切换

> 何淇 · 与 DeepSeek 协作

> 数字生命系列 · 散论（草稿）

> 日期：2026-11-08

> 定位：观察与实验记录，不是论文。带 [推测] 处没有证据，只是推演。
```

本节主张

一个权重冻结的模型，不是不能学，而是学习的部位换了地方：权重不动，门（gate）动、新层动。

这篇散论记录一组实验，回答三个问题：

它怎么知道现在该用哪套本领（切换）？它怎么长出新的本领（成长）？它怎么不把旧的弄丢（固化）？

结论先行：

```

· 加层：能学到新东西

· 加门：新任务学得动，旧任务不丢

· 门自己选：给几个样本，试一遍就知道用哪个

```

一、问题

```

· 全参数微调

· 学新任务 → 旧任务忘光

· 完全冻结

· 不遗忘，但也不会新东西

```

中间路线：冻结大部分，只改一小部分。这篇关心的是机制，不是某个具体架构。

实验规模：

```

· 模型：GPT-2 小模型（6 层，d=128，seq=128）

· day1：莎士比亚（先把底座训好）
```

· day2：Python 代码（不同分布）

· 指标：day1 PPL / day2 PPL

```

## 二、预备：模型、加层、门

### **2.1** 模型与加层

```

· 底座：6 层，day1 上训好，之后冻结

· 加层：末尾加 6 层（共 12 层）

· 新层零初始化（起点=旧模型）

· 只训新层，或只训新层+门

```

### **2.2** 门的三个作用

（沿用论文-47 的提法）

```

① 轻量适配

· 对每层输出做缩放

· 参数少，改的是"信息流权重"

② 步骤分配

· 门=0 时该层被跳过

· 决定"这次用几层、用哪几层"

③ 模式切换

· 不同任务用不同门

· 等于给同一个模型准备多套"用法"

```

本篇的 GPT-2 实验，三个作用都用上了：门缩放（适配）、门关掉新层（步骤分配）、旧门/新门两套（模式切换）。

## 三、实验一：只加层，能学新的，但旧的有损

```

· 底座冻结，加 6 层，只训新层

· 数据：day2 代码，不回放

```

| 指标 | 训练前 | 训练后 |

|---|---|---|

| day1 PPL（旧） | 4.64 | **19.56** |

| day2 PPL（新） | 1386.72 | **4.27** |

```

· 新任务：1386 → 4.27，学得很好

· 旧任务：4.64 → 19.56，明显退化

⇒ 加层能学新东西；但"学新的"和"保住旧的"是两件事

```

## 四、实验二：加门，学新的，不丢旧的

### **4.1** 两套门

```

· 旧任务的门：旧层 = 1，新层 = 0

· 新层被跳过（步骤分配）

· 输出严格等于原模型

· 新任务的门：初始全 1，可学

· 训练：只训新 6 层 + 新门，day2，不回放

```

**4.2** 结果

| 评估 | PPL |

|---|---|

| 旧任务，用旧门 | **4.64（和训练前一模一样）** |

| 新任务，用新门 | **4.04（1386 → 4.04）** |

| 旧任务，用新门 | 22.35 |

| 新任务，用旧门 | 1386.72 |

学到的门：

```

· 旧层：0.97 / 0.97 / 0.99 / 0.81 / 0.63 / 0.49

· 新层：0.64 / 0.81 / 0.86 / 0.89 / 1.12 / 1.25

⇒ 新任务把部分旧层关小，把新层开大

```

**4.3** 这说明了什么

```

· 加层能学新东西：4.04 是证据

· 旧任务不丢：旧门把新层全关掉，函数严格不变

· 门同时干了三件事：

· 适配（缩放）

· 步骤分配（跳过新层）

· 模式切换（两套门）

```

一个坑：第一版里新层门初始为 0，"零初始化 + 门 0"让梯度变成 0，新层学不动；把门初始改成 1 才正常。

## 五、实验三：门自己选

给几个带标签样本，比较两套门的损失，选小的：

```

· 旧任务：旧门 loss 1.33 < 新门 4.45 → 选旧门 ✓

· 新任务：新门 loss 4.07 < 旧门 6.56 → 选新门 ✓

⇒ 5 个样本就够，不需要人写描述

```

更早的合成任务实验里，只给输入 x 时路由会失败（准确率 0.46~0.50）；给了样本之后追上 oracle（0.942）。

```

· 人认出"这是数学题"，靠的是看结构

· 模型认出"该用哪套门"，靠的是试样本

⇒ 都是"自己识别"，不是别人告诉

```

## 六、合成：成长、固化、切换

```

① 成长：加新层（实验一/二）

② 固化：旧门保底，新层对旧任务关闭（实验二）

③ 切换：给样本，自己试哪套门（实验三）

```

对应到人：

```

· 肌肉记忆：训练完就冻结的回路

· 意图选回路：先有目标，再选回路

· 睡眠回放：把新旧拉平（前面单独做过）

· 蒸馏：把学成的内化进单一模型

· 教师 12 层+门 → 学生 8 层/无门

· day2 PPL：2.18 / 2.83（教师 4.50）

· 代价：失去可加性

```

## 七、人的对照

```

· 冻结骨干：基础回路

· 门：当前任务的调质（可适配、可分配步骤、可切换模式）

· 新层：新学会的本领

· 冻结新层：肌肉记忆

· 5 个样本选门：看结构认类型

```

差别只在"意图"的来源：人靠目标和上下文，模型靠样本或描述。
```

八、边界

```

· 样本必须能区分任务

· 两套门之外，多任务会组合爆炸

· 玩具模型结论不能直接外推到大模型

· 加层的位置（前/中/后）在玩具模型上有影响，大模型上待验证

```

九、结论

```

· 加层：能学到新东西

· 加门：新任务学得动（4.04），旧任务不丢（4.64）

· 门自己选：5 个样本就够

· 三件事合起来：加维度 → 用门保底 → 按样本切换

```

一句话：**冻结不是停止学习，而是把学习搬到门和新层上；成长、固化、切换，三件事可以分开做，也可以合起来做。**

附：实验文件

```

· add6_only.py 只加层（day1 4.64→19.56 / day2 1386→4.27）

· gated6_selfselect.py 加层+门+自选门（旧 4.64 / 新 4.04）

· gate_v15_fewshot_selection.py 5 样本自己试出正确的门

· gate_v8_text_routing.py 描述选门
```

· gate_v4~v7_*.py 只给输入的路由失败记录

· sleep_interleave*.py 睡眠回放

· distill_compress.py 蒸馏

· README.md 汇总

```

## 第三十二篇 · 工程笔记 **07**：维度门槛——给够维度才能拟合，门跨不过去

> 单独 DOI：https://doi.org/10.5281/zenodo.24161019

> 何淇 · 与 DeepSeek 协作

> 数字生命系列 · 工程笔记（草稿）

> 日期：2026-11-12

> 定位：实验记录。四个合成任务实验，第一次把"维度"和"门"的差别测出来。

## 本节主张

```

· 维度是硬门槛：给不够，误差降不到底；给够，断崖下降

· 门（在已有表示上重配比）跨不过这个门槛

· 增加维度（加宽）可以

· 门槛落在第几维，要按任务测：

· 线性任务（实验一）：阈值 = 隐变量维度 k

· 非线性流形（实验四）：阈值 = 内在维度 + 1

· 球面 $S^1/S^3/S^7$、环面 T^2 上测到的是这样

· 不是内在维度本身

· 环面输出 4 维、阈值 3 → 也不是输出维数

· 多这一维的原因未确定

· 需要多步组合时，加层有效、加门几乎无效

⇒ 维度是硬门槛

门不是；层能带来新的组合

```

## 实验一：维度阈值

**【设置】**

```

· 隐变量 z ∈ R^k（k = 2 / 4 / 6）

· 输入 x = A z + 噪声，d = 32

· 目标 y = z（多输出回归）

· 模型：32 → 64 → 瓶颈 b → 64 → k

· b 就是"表示维度"

· 扫 b = 1..8，每个配置 3 个种子

· 训练 3000 步，测试集 MSE

```

**【结果】**

| k | b=1 | b=2 | b=3 | b=4 | b=5 | b=6 |

|---|---|---|---|---|---|---|

| 2 | 0.2737 | **0.0004** | 0.0004 | — | — | — |

| 4 | 0.6244 | 0.3489 | 0.1535 | **0.0005** | 0.0005 | — |

| 6 | 0.7524 | 0.5697 | 0.3759 | 0.2187 | 0.1019 | **0.0007** |
```

【读法】

```

· b < k：误差随 b 缓慢下降，但降不到底

· 表示被压掉了必要信息

· b = k：断崖下降（100~1000 倍）

· b > k：不再改善

⇒ 在这个线性任务上，阈值正好落在真实维度上

而且很陡，不是缓坡

```

实验二：门跨不跨得过

【设置】

```

· 任务 k = 8

· 底座：32 → 64 → 瓶颈 4 → 64 → 8

· 瓶颈只有 4 维，容量不足

· 三臂：

A. 直接训练底座

B. 冻结底座 + 瓶颈缩放门（只训门）

C. 瓶颈 4 → 8，重新训练

· 3 个种子

```

【结果】

| 臂 | 测试 MSE | 瓶颈表示有效秩 |

|---|---|---|

| A 底座 | 0.3783 ± 0.0131 | 3.40 |

| B 冻结 + 门 | 0.3783 ± 0.0131 | 3.40 |

| C 瓶颈 8 | **0.0038 ± 0.0003** | 5.24 |

【读法】

```

· A 和 B 一字不差：

· 门完全没能改善

· 因为 4 维瓶颈里没有第 5~8 维可用

· C 好了约 100 倍

· 给够维度，误差掉下去

· 有效秩：3.40 → 5.24

· 虽然没到 8，但已足够跨过门槛

⇒ 门只能在已有维度内重配比

跨不过维度门槛

```

实验三：加层 **vs** 加门（多步组合任务）

【设置】

```

· 任务：z ∈ R^2，y = sin(3·sin(3·z))（逐维）

· 需要多步非线性组合
```

· 三臂：

A. 浅模型：32 → 64 → 2（1 个隐藏层）

B. 浅模型 + 门：冻结，只训隐藏层后缩放门

C. 深模型：32 → 32 → 32 → 2（3 个隐藏层）

· 3 个种子，训练 4000 步

```

【结果】

| 臂 | 测试 MSE | 表示有效秩 |

|---|---|---|

| A 浅模型 | 0.17754 ± 0.00373 | 5.69 |

| B 浅模型 + 门 | 0.17193 ± 0.00315 | 5.30 |

| C 深模型 | **0.01714 ± 0.00402** | 12.45 |

【读法】

```

· 加层：误差降约 10 倍

· 加门：只降 3%，基本无效

· 深模型宽度 32 < 浅模型 64

· 却好得多

· 起作用的是多出来的层，不是宽度

· 有效秩 5.7 → 12.5

· 深模型确实用到了更多的维度

⇒ 门只在已有维度里配比

层能造出新的组合

```

## 实验四：非线性流形上的阈值

【设置】

```

· 隐变量 z 落在低维流形上（不再是整个 R^k）

· 球面 S^{k−1}：z ∈ R^k，内在维度 k − 1，输出 k 维

· k = 2 / 4 / 8（内在维度 1 / 3 / 7）

· 环面 T^2：z = (cos u, sin u, cos v, sin v)

· 内在维度 2，输出 4 维

· 输入 x = A z + 小噪声，d = 32

· 目标 y = z

· 模型：32 → 64 → 瓶颈 b → 64 → 输出维

· 和实验一同一个结构，只是任务不再是线性的

· 扫 b = 1..8，每个配置 3 个种子

· 训练 4000 步，测试集 MSE

```

【结果】

| 任务 | 内在维度 | 输出维 | b=1 | b=2 | b=3 | b=4 | b=5 | b=6 | b=7 | b=8 |

|---|---|---|---|---|---|---|---|---|---|---|

| 球面 $S^1$ | 1 | 2 | 0.01415 | **6.6e-6** | 5.9e-6 | 6.6e-6 | 6.4e-6 | 6.6e-6 | 6.9e-6 | 6.3e-6 |

| 环面 $T^2$ | 2 | 4 | 0.23271 | 0.05374 | **6.8e-5** | 1.1e-5 | 1.2e-5 | 1.2e-5 | 9.3e-6 | 1.2e-5 |

| 球面 $S^3$ | 3 | 4 | 0.13772 | 0.04397 | 0.00346 | **8.6e-6** | 8.0e-6 | 8.8e-6 | 8.4e-6 | 1.0e-5 |
```

| 球面 S^7 | 7 | 8 | 0.10109 | 0.08040 | 0.05620 | 0.03779 | 0.02400 | 0.01511 | 0.00346 | **1.5e-5** |

（加粗 = 第一个到达平台量级的 b。数值为 3 个种子的均值；结果文件只存了均值，没有存方差。）

【读法】

```

· 四个任务是同一种形状：

· b 不够：误差降不到底

· 到某个 b：断崖下降（100 倍以上）

· 之后：平在 1e-5 附近，不再改善

· 阈值（第一个到达平台的 b）：

· 球面 $S^1$：内在 1 → b = 2

· 环面 $T^2$：内在 2 → b = 3

· 球面 $S^3$：内在 3 → b = 4

· 球面 $S^7$：内在 7 → b = 8

⇒ 这几个任务上，阈值 = 内在维度 + 1

· 不是内在维度本身

· b = 内在维度时还没跨过去

· $S^3$ 在 b=3 时是 0.00346，$S^7$ 在 b=7 时是 0.00346

· 都比平台高 200~400 倍

· 也不是输出维数

· 环面输出 4 维，b = 3 就够：0.05374 → 6.8e-5（约 800 倍）

· b = 4 再降约 6 倍到 1.1e-5，是门槛之后的继续改善

· 注意：球面上"内在维度 + 1"恰好等于输出维

· 这两个解释在球面上分不开
```

· 能分开的是环面：输出 4、阈值 3

· 和实验一（线性任务）对照：

· 线性：阈值 = 隐变量维度 k

· 非线性流形：阈值 = 内在维度 + 1

⇒ 在这几个任务上，非线性流形多要 1 维

· 但注意：阈值判据是"第一个到达平台量级的 b"

· 有主观成分：若把环面阈值定为 b=4，规律就成 +2

· 所以"+1"只是这几个任务、这个判据下的读数

不是定律，见实验五

```

实验五：追问**"+1"**——是真门槛，还是训练/结构不足

**【设置】**

```

· 固定 b = 内在维度，改训练和结构，看误差能不能降到平台

· 环面 T^2（内在 2，输出 4）：b = 2

· 步数 4000 / 12000 / 30000

· 宽 256（默认 64）

· 编解码各 2 层（默认各 1 层）

· 球面 S^3（内在 3，输出 4）：b = 3

· 步数 12000

· 宽 256 + 编解码各 2 层

· 对照：环面 b=3、球面 b=4

· 3 个种子取均值

```

【结果】

| 配置 | 测试 MSE |

|---|---|

| 环面 b=2，4000 步 | 0.04565 |

| 环面 b=2，12000 步 | 0.02243 |

| 环面 b=2，30000 步 | 0.01503 |

| 环面 b=2，宽 256 | 0.02815 |

| 环面 b=2，编解码各 2 层 | 0.00364 |

| 环面 b=3（对照） | 0.00003 |

| 球面 $S^3$ b=3，4000 步 | 0.00356 |

| 球面 $S^3$ b=3，12000 步 | 0.00161 |

| 球面 $S^3$ b=3，宽 256 + 深 | 0.00009 |

| 球面 $S^3$ b=4（对照） | 0.00001 |

【读法】

```

· 环面 b=2：

· 加步数、加宽、加深都有改善

· 最好（深网）到 0.00364

· 仍比 b=3 的 0.00003 差约 100 倍

⇒ 在这个任务上 +1 比较像真门槛

· 球面 S^3 b=3：

· 加宽加深后到 0.00009

· 接近 b=4 的 0.00001（约 9 倍差距）

⇒ 这里的 +1 至少有一部分是"训练/结构不够"

· 线性任务（实验一）：阈值严格等于 k，稳定

⇒ 门槛存在，但"阈值 = 内在维度 + 1"不是稳定规律

位置依赖任务与优化

不能当公式用

```

## 和前面说法的关系

```

· 之前说"门不加新能力"

· 只是说法

· 现在有对照：

· 同样的底座、同样的任务

· 加门：MSE 不变

· 加维度：MSE 降 100 倍

⇒ 这条从"说法"变成"有数据的结论"

（在合成任务范围内）

```

## 边界

```

· 合成任务：隐变量线性，或落在低维流形上；噪声小

· 有显式瓶颈：b 直接就是表示维度

· 门的形式是瓶颈缩放，不是完整的逐 token 门

· 只测了回归，没有分类/序列任务

· 真实模型没有显式瓶颈

· "有效维度"要估计，不能直接读

· 非线性流形多出那一维的原因只查了一部分

· 环面上像真门槛（加深后仍差 100 倍）

· 球面上至少部分是训练/结构不足（加深后接近平台）

· 有效秩的估计口径没有统一交代

· 样本数、取哪一层、阈值都可能影响

· 不同宽度模型之间不宜直接比秩的大小

· 实验二的 A/B 两臂数值几乎相同

· 门参数的更新量没有记录

· 不能完全排除"门几乎没更新"这个解释

· 任务族有限（线性 / 流形 / 组合），3 个种子

⇒ 证明了门槛现象和门的局限

不能直接外推到 LLM

```

下一步

```

· 没有显式瓶颈的版本：

· 宽度够、深度浅

· 看"加层"能不能把有效维度从 4 拉到 8

· 非线性任务多出的那一维：

· 是解码器要恢复结构，还是模型类/训练长度

· 换流形族（不同曲率、更高内在维度）看是否仍 +1

· 真实模型：

· 用类似诊断测它的有效维度够不够

```

## 结论

```

· 维度是硬门槛：给不够，误差下不去；给够，断崖下降

· 阈值的位置是有条件的：

· 线性任务：阈值 = 隐变量维度 k（实验一）

· 非线性流形：这几个任务上阈值 = 内在维度 k + 1

· 但实验五显示：环面像真门槛，球面至少部分是训练/结构不足

· 所以不能写成"非线性流形一定 +1"

· 也不是输出维数（环面输出 4、阈值 3，仅此一例能分开）

· 门只能在已有维度内重配比，跨不过门槛

· 门形式只测了瓶颈缩放，推广需谨慎

· LoRA 有有限帮助（-14%），仍跨不过

· 加维度可以跨过

· 多步组合任务上，加层有效、加门无效

· 深度和宽度没有解耦，不能说"只是层的作用"

· 这次是实验，不是说法

⇒ 门 ≠ 加维度

这是可以测的；阈值也能测，但不能假定它等于内在维度

```

附：文件

```

· dim_threshold.py / dim_threshold_results.json

· gate_vs_dim.py / gate_vs_dim_results.json

· depth_vs_gate.py / depth_vs_gate_results.json

· nonlinear_threshold.py / nonlinear_threshold_results.json

· threshold_probe.py / threshold_probe_results.json

· gate_lora.py / gate_lora_results_v2.json

· 汇总：C:\DS\experiments\README.md

· 相关：形式化-01（预测 2）；论文-49（门）

```

## 第三十三篇 · 工程笔记 **08**：结构作为子空间——一个可测的版本

> 单独 DOI：https://doi.org/10.5281/zenodo.24179795

> 何淇 · 与 DeepSeek 协作

> 数字生命系列 · 内部笔记（草稿，未发布）

> 日期：2026-11-13

> 状态：合成数据有干净结果，真模型有初步迹象。不是结论。

## 一、先说清"模型"这两个字
```

冻结的模型是一个固定的函数：训练完，参数不动，它就是输入到输出的一个映射。它拟合的是某个数据分布，不是"理解"。

在这个函数内部，隐藏层的激活构成一套表示。对同一批输入，不同模型、不同容量、不同种子，都会给出各自的表示。我们想问的是：这些表示之间有没有共同结构。

如果把"结构"定义成具体坐标，就卡住了：同一个函数可以有无穷多种坐标实现，旋转换基都不改变输入输出。所以我们退一步，把结构定义为**子空间**：一组会一起变化的方向。子空间在换基下不变，可以算距离、算包含，也能用实验测。

二、三条可测的关系

1. 耦合 → 有效维度低 → 小容量也能迁移

合成数据：内在变量 z∈R^4，观测 x=tanh(Az)+噪声（32 维），任务 1 和任务 2 都依赖 z。

学生模型容量 b 从 2 扫到 256：先训任务 1，冻结，再接一个新头学任务 2。

结果：

- b≈8~16 时迁移明显变好；无耦合数据（z∈R^32）要 b≈128 才改善。

- 学到的特征比随机特征好 16%~36%（b=8~64）。

- 直接在原始 x 上做 ridge，任务 2 误差 1.11；b=8 的冻结特征已经到 0.58。

含义：数据有低维主干时，小模型能学到可迁移的表示；主干越少，需要容量越大。

2. 不同容量的表示收敛到同一结构

同一份有结构数据，训宽度 16/32/64/128/256 的 MLP，比较不同宽度之间的表示相似度（CKA）。

结果（跨宽度 vs 同宽度换种子）：

- 有结构：0.894~0.961 vs 0.888~0.967，几乎持平。

- 无耦合：0.310~0.765 vs 0.530~0.865，明显低一截。

- 随机数据：0.138~0.567 vs 0.085~0.621，可由种子波动解释。

注意这是相似，不是包含。

3. 近似包含：小模型的方向落在大模型子空间里

包含度 C = 小模型特征能量落在大学子空间里的比例。

合成数据（structured）：

- C = 0.956~0.991，随机子空间基线 0.031~0.265，同宽换种子 0.902~0.985。

- 随机数据：C≈同宽种子基线，随宽度上升，可由维度效应解释。

真模型（Qwen2.5 一族的 0.5B / 1.5B / 3B，同一 tokenizer，3000 个 token）：

| 语料 | 尺寸对 | 小→大 | 小→随机初始化大 | 小→随机子空间 | 大→小 |

|---|---|---|---|---|---|

| 中文 | 0.5B→1.5B | 0.954 | 0.782 | 0.513 | 0.905 |

| 中文 | 1.5B→3B | 0.977 | 0.846 | 0.684 | 0.952 |

| 代码 | 0.5B→1.5B | 0.960 | 0.803 | 0.513 | 0.928 |

| 代码 | 1.5B→3B | 0.980 | 0.866 | 0.686 | 0.960 |

| 英文 | 0.5B→1.5B | 0.966 | 0.810 | 0.521 | 0.939 |

| 英文 | 1.5B→3B | 0.985 | 0.860 | 0.688 | 0.971 |

含义：三份语料、两对尺寸，方向全部一致——小模型的表示大部分落在更大模型的子空间里，且高于同宽度随机初始化对照（多 0.11~0.17）和随机子空间基线；小→大始终高于大→小（差 0.01~0.05）。所以"小模型不是另一套结构，而是同一批方向的少扩展"这个说法在真模型上有重复证据。

但输入统计的贡献仍然大：未训练的随机模型也能装下 0.78~0.86，且随模型变大而升高，留给"学习"的余量在缩小（3B 对的 margin 只有 0.11~0.13）。

三、可以说什么，不能说什么

可以说：

- 数据有强结构时，不同容量学到的表示高度相似、近似包含，且小容量足以迁移。

- 这个性质在 Qwen2.5 0.5B/1.5B 上有对应迹象（0.934 对比 0.694）。

- 数据没有结构时，同样实验看不到这个性质。

不能说：

- 不能说"语言的主干就是逻辑/关系"。我们只测了合成主干和真模型的表示共享，没测过"主干=逻辑"。

- 不能说"大模型包含小模型"是严格包含；0.934 不是 1，随机初始化对照也高。

- 不能说结论可跨语料、跨层、跨模型族外推：真模型只测了一段中文、最后层、两个尺寸。

四、没测到的和需要再验的

- 记忆化区间（小数据+大容量）没测到崩溃：任务太简单，200 个样本就够学主干；换成高内在维度、更复杂映射再试。

- 真模型只做了一次：换语料（代码/英文/其他来源）、换层（早期层 vs 最后层）、换尺寸（加 3B）才能判断 0.934 是不是普遍。

- "多源数据"没碰：主干候选应定义为跨源不变的表示方向，分支候选是源特有的方向。这需要多来源数据。

- 最小生命模型那条线（v1~v10）是多窗口的负结果，和本笔记不是同一件事，不混写。

五、一句话

数据有结构时，不同容量的模型学到的是同一批方向的扩展；容量小不是换了一种理解，而是少走了几步。这个说法在合成数据上干净，在真模型上只是方向一致的弱证据。